

Ex. No: 2 Illustrate UNIX Commands and Shell Programming

1: Greatest Among Three Numbers

Program:

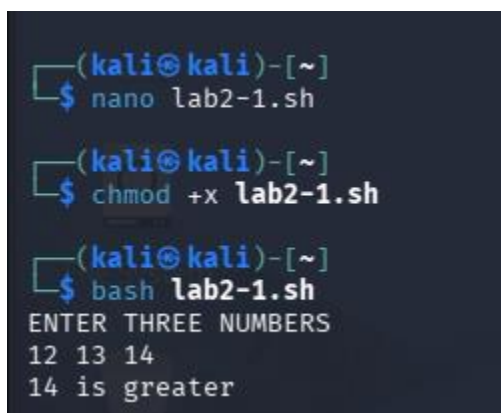
```
#!/bin/bash

echo "ENTER THREE NUMBERS"

read a b c

if [ $a -gt $b ] && [ $a -gt $c ]
then
echo "$a is greater"
elif [ $b -gt $c ]
then
echo "$b is greater"
else
echo "$c is greater"
fi
```

Output:

A terminal window on a Kali Linux system. The prompt is (kali@kali)-[~]. The user enters 'nano lab2-1.sh' to create the script. Then, they enter 'chmod +x lab2-1.sh' to make it executable. Finally, they enter 'bash lab2-1.sh' to run it. The script prompts 'ENTER THREE NUMBERS' and the user enters '12 13 14'. The script then outputs '14 is greater'.

```
(kali@kali)-[~]
$ nano lab2-1.sh

(kali@kali)-[~]
$ chmod +x lab2-1.sh

(kali@kali)-[~]
$ bash lab2-1.sh
ENTER THREE NUMBERS
12 13 14
14 is greater
```

2. Factorial of a Given Number

Program:

```
#!/bin/bash

echo "ENTER THE NUMBER:"

read n

fact=1

while [ $n -gt 1 ]

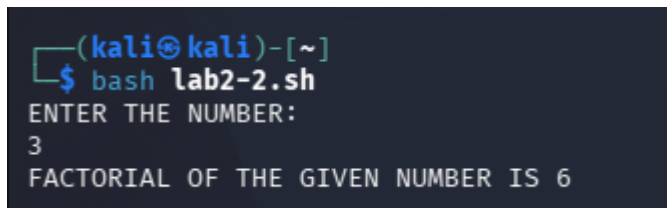
do

fact=$((fact * n))

n=$((n - 1))

done
```

Output:

A terminal window with a dark background. The prompt is '(kali㉿kali)-[~]'. The user has entered 'bash lab2-2.sh'. The script prompts 'ENTER THE NUMBER:' and the user has entered '3'. The script outputs 'FACTORIAL OF THE GIVEN NUMBER IS 6'.

```
(kali㉿kali)-[~]
$ bash lab2-2.sh
ENTER THE NUMBER:
3
FACTORIAL OF THE GIVEN NUMBER IS 6
```

3. Sum of Odd Numbers up to N

Program:

```
#!/bin/bash

echo "ENTER THE RANGE:"

read n

x=1

sum=0

while [ $x -le $n ]

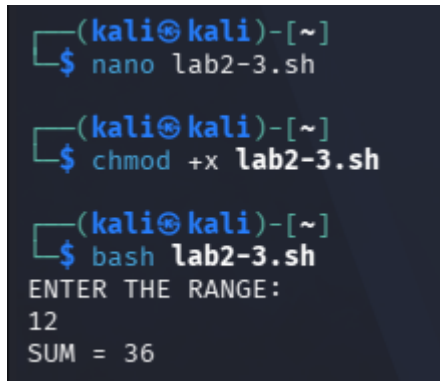
do
```

```
sum=$((sum + x))
```

```
x=$((x + 2))
```

```
done
```

Output:



```
(kali㉿kali)-[~]  
$ nano lab2-3.sh  
  
(kali㉿kali)-[~]  
$ chmod +x lab2-3.sh  
  
(kali㉿kali)-[~]  
$ bash lab2-3.sh  
ENTER THE RANGE:  
12  
SUM = 36
```

4. Generation of Fibonacci Numbers

Program:

```
#!/bin/bash
```

```
echo "ENTER THE LIMIT:"
```

```
read n
```

```
p=-1
```

```
q=1
```

```
i=1
```

```
while [ $i -le $n ]
```

```
do
```

```
r=$((p + q))
```

```
p=$q
```

```
q=$r
```

```
echo "$r"
```

```
i=$((i + 1))
```

done

Output:

```
(kali㉿kali)-[~]  
$ bash lab2-4.sh  
ENTER THE LIMIT:  
12  
0  
1  
1  
2  
3  
5  
8  
13  
21  
34  
55  
89
```

5.Arithmetic Calculator

Program:

```
#!/bin/bash
```

```
echo "ENTER THE VALUE OF A:"
```

```
read a
```

```
echo "ENTER THE VALUE OF B:"
```

```
read b
```

```
echo "ENTER THE OPTION TO PERFORM"
```

```
echo "1. ADDITION"
```

```
echo "2. SUBTRACTION"
```

```
echo "3. MULTIPLICATION"
```

```
echo "4. DIVISION"
```

```
read op
```

```
case "$op" in
```

```
1) echo "Result = $((a + b))" ;;
2) echo "Result = $((a - b))" ;;
3) echo "Result = $((a * b))" ;;
4) echo "Result = $((a / b))" ;;
*)
```

Output:

```
(kali㉿kali)-[~]
$ nano lab2-5.sh

(kali㉿kali)-[~]
$ chmod +x lab2-5.sh

(kali㉿kali)-[~]
$ bash lab2-5.sh
ENTER THE VALUE OF A:
12
ENTER THE VALUE OF B:
23
ENTER THE OPTION TO PERFORM
1. ADDITION
2. SUBTRACTION
3. MULTIPLICATION
4. DIVISION
1
Result = 35
```

6.Largest Digit of Number

Program:

```
#!/bin/bash

echo "ENTER THE NUMBER"

read a

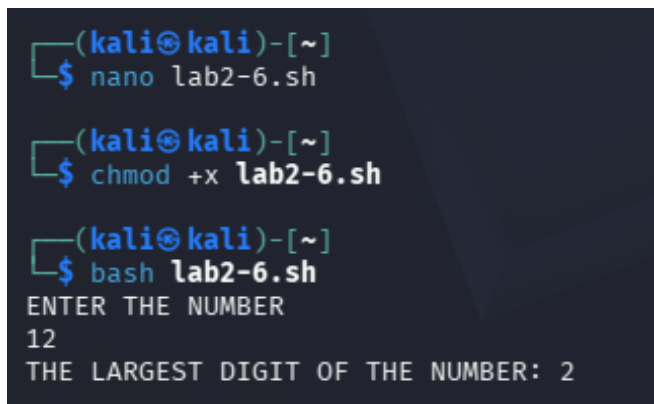
max=0

while [ $a -gt 0 ]

do
```

```
r=$((a % 10))  
if [ $r -gt $max ]  
then  
max=$r  
fi  
a=$((a / 10))  
done  
echo "THE LARGEST DIGIT OF THE NUMBER: $max"
```

Output:



```
(kali㉿kali)-[~]  
$ nano lab2-6.sh  
  
(kali㉿kali)-[~]  
$ chmod +x lab2-6.sh  
  
(kali㉿kali)-[~]  
$ bash lab2-6.sh  
ENTER THE NUMBER  
12  
THE LARGEST DIGIT OF THE NUMBER: 2
```

7. Palindrome String Check

Program:

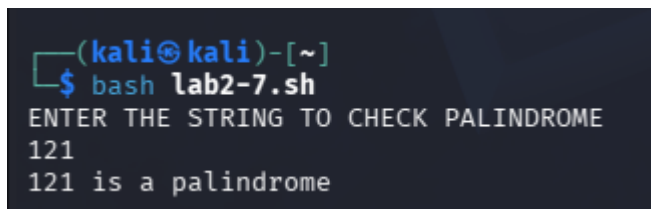
```
#!/bin/bash  
  
echo "ENTER THE STRING TO CHECK PALINDROME"  
  
read str  
  
len=$(echo -n "$str" | wc -c)  
  
i=1  
  
j=$((len / 2))
```

```

while [ $i -le $j ]
do
k=$(echo "$str" | cut -c $i)
l=$(echo "$str" | cut -c $len)
if [ "$k" != "$l" ]
then
echo "$str is not a palindrome"
exit
fi
i=$((i + 1))
len=$((len - 1))
done
echo "$str is a palindrome"

```

Output:



```

(kali㉿kali)-[~]
$ bash lab2-7.sh
ENTER THE STRING TO CHECK PALINDROME
121
121 is a palindrome

```

8. Reverse of a Given Number

Program:

```

#!/bin/bash

echo "ENTER THE NUMBER"

read n

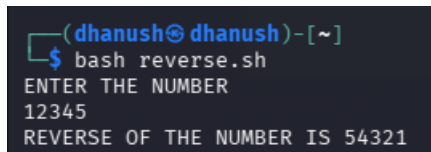
rnum=0

while [ $n -ne 0 ]

```

```
do
remainder=$((n % 10))
rnum=$((rnum * 10 + remainder))
n=$((n / 10))
done
echo "REVERSE OF THE NUMBER IS $rnum"
```

Output:



```
(dhanush@dhanush)-[~]
$ bash reverse.sh
ENTER THE NUMBER
12345
REVERSE OF THE NUMBER IS 54321
```

A terminal window with a dark background. The prompt is '(dhanush@dhanush)-[~]'. The user enters 'bash reverse.sh'. The script prompts 'ENTER THE NUMBER'. The user enters '12345'. The script outputs 'REVERSE OF THE NUMBER IS 54321'.